

CC3001 Algoritmos y Estructuras de Datos**Profesores:** Nelson Baloian, Patricio Poblete, e Iván Sipirán**Auxiliares:** Sebastián Acuña, Daniela Espinoza, Cristián Llull Torres,

Benjamín Osses, Anish Samtani, y Marcelo Zamorano

**Auxiliar 6**

29 de mayo de 2026

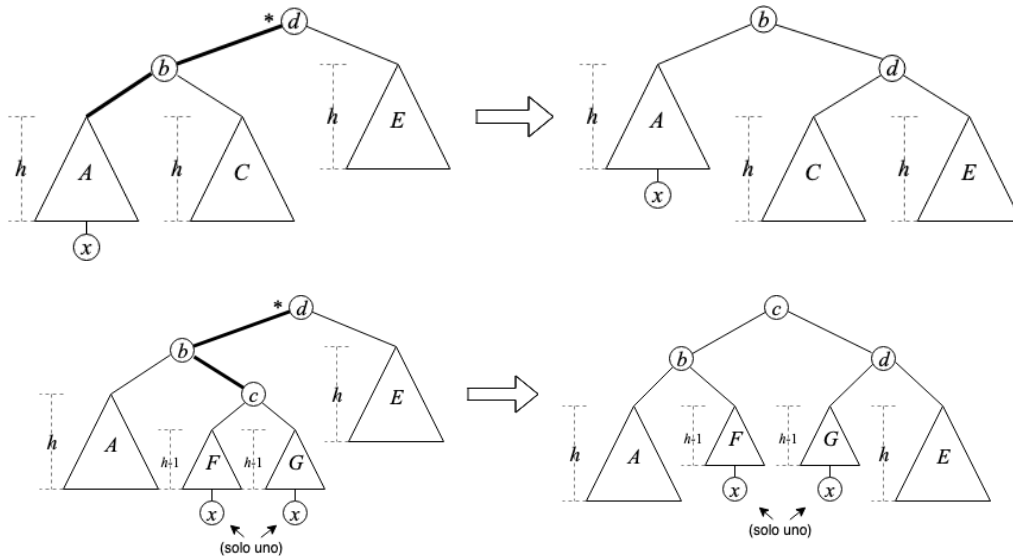
P1. Inserción y eliminación en árboles AVL y 2-3

Considere las siguientes secuencias de inserciones y eliminaciones, donde un + significa una inserción y un - una eliminación.

- (a) +1 +2 +3 +4
- (b) +1 +2 +3 -2 +4 +5 -1 +6
- (c) +1 +3 +2 +6 +5 +4 -1 -2
- (d) +25 +10 +32 +27 +74 +48
- (e) +25 +10 +32 +27 +74 -27

Muestre, en un diagrama, el estado del árbol luego de cada operación para un árbol 2-3 y para un árbol AVL.

Diagrama de rotaciones en un árbol AVL:

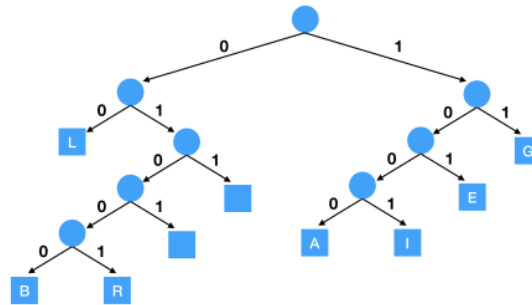
**P2. Implementación de Tries**

Un trie es un tipo de árbol de búsqueda, que tiene la particularidad de no guardar valores en sus nodos, sino que la información está dada por la posición que tiene un nodo dentro del árbol.

La búsqueda se realiza desde la raíz, eligiendo la i -ésima arista a recorrer según i -ésimo carácter del valor que se desea buscar. Note que la búsqueda tarda $O(\text{largo de la palabra buscada})$.

Según el alfabeto que se escoja se puede construir distintos tipos de tries; Por ejemplo el trie con el alfabeto $\Sigma = \{0, 1\}$ se dice que es un árbol digital y cada palabra ingresada debe tener todos sus caracteres en Σ . En lo que sigue, ocuparemos la codificación binaria de nuestro alfabeto latino ($\{A, B, \dots, Z\}$) dada por el código ASCII como entrada del trie.

A continuación se muestra un ejemplo de trie, al que se le insertaron las codificaciones ASCII de las letras A, B, E, I, G, L y R:



Dada la siguiente definición de nodo de Trie:

```
import numpy as np

class NodoiTrie:
    def __init__(self, izq = NodoeTrie(), der = NodoeTrie()):
        self.info = ''
        self.izq = izq
        self.der = der

class NodoeTrie:
    def __init__(self, info = None):
        self.info = info

class Trie:
    def __init__(self, raiz = NodoeTrie()):
        self.raiz = raiz
```

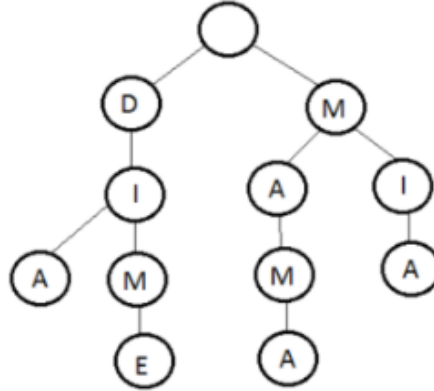
Se le pide lo siguiente:

- Implemente una función de inserción, que luego de insertar imprima el camino en el árbol hasta la posición de inserción.
- Implemente una función de búsqueda, que entregue una tupla con el valor booleano si el elemento está en el árbol y el camino correspondiente.
- Propuesto:** Pruebe su implementación ingresando las secuencias binarias de las letras A, B, E, I, G, L, R en formato ASCII.
- Propuesto:** Tal como se menciona en el enunciado, se podría modificar el trie de modo que procese un alfabeto distinto. Discuta como implementar esta idea para ingresar secuencias

de palabras del español. Además, ¿cómo se podría solucionar el caso en que se ingresa una palabra que es prefijo de otra?

P3. Árboles de búsqueda digital

Los árboles de búsqueda digital (ABD) son un híbrido entre los ABBs y los tries. Las llaves se almacenan en los nodos internos, al igual que en un ABB, pero la ramificación se hace según el diccionario que se este utilizando. La búsqueda se realiza desde la raíz, eligiendo la i -ésima arista a recorrer según i -ésimo carácter del valor que se desea buscar.



(a) Inserte en un ABD vacío las palabras PANADERÍA, PAN, PANADERO, PANCITO, PALA, PALABRERÍA, PALABRA. ¿Cuál es el prefijo más largo que se repite?

(b) Programe una función `buscar_prefijo` que reciba un nodo a un ABD y un string prefijo. La función debe imprimir todas las palabras del ABD que comiencen con dicho prefijo.